MMUKO-OS — NSIGII Heartfull Firmware Compositor

OBINexus Computing | **Nnamdi Michael Okpala Version:** 0.1-DRAFT | 20 March 2026 **Status:** Active specification

---

What this is

MMUKO-OS is a **constitutional computing environment** built on the principle that a computer should not boot until the operator's fundamental needs are verified. The NSIGII Heartfull Firmware is the pre-boot calibration layer that enforces this — it runs before any OS process, application, or UI loads.

The firmware implements a **Maslow needs-gate**: Tier 1 (physiological) and Tier 2 (safety) must be satisfied or actively pending before computation proceeds. This is not a metaphor. It is a boot protocol.

Quick start

1. Assemble the boot sector

```powershell
# Windows (NASM must be installed: https://nasm.us)
nasm -f bin boot.asm -o boot.bin

# Verify: should be exactly 512 bytes with 0xAA55 at offset 510
```

2. Write the disk image

```powershell
# PowerShell — write boot sector to image
$boot = [System.IO.File]::ReadAllBytes("boot.bin")
$img  = [System.IO.File]::ReadAllBytes("mmuko-os.img")
[System.Array]::Copy($boot, 0, $img, 0, 512)
[System.IO.File]::WriteAllBytes("mmuko-os.img", $img)
```

3. Boot in QEMU

```powershell
qemu-system-x86_64.exe -drive format=raw,file=mmuko-os.img
```

You should see:

```
MMUKO-OS NSIGII v0.1
Y=1 N=0 M=-1
[N]Need
[S]Safe
[I]Ident
[G]Gov
[I]Probe
[I]Integ

NSIGII_VERIFIED
HR:PASS
BOOT_OK
```

4. Build the C firmware library (WSL or Linux)

```bash
# From the project directory
mkdir -p build/obj build/lib

gcc -std=c11 -Wall -fPIC -O2 -c heartfull_membrane.c   -o build/obj/heartfull_membra
gcc -std=c11 -Wall -fPIC -O2 -c bzy_mpda.c             -o build/obj/bzy_mpda.o
gcc -std=c11 -Wall -fPIC -O2 -c tripartite_discriminant.c -o build/obj/tripartite_di

gcc -shared -o build/lib/libnsigii_firmware.so \
    build/obj/heartfull_membrane.o \
    build/obj/bzy_mpda.o \
    build/obj/tripartite_discriminant.o -lm

ar rcs build/lib/libnsigii_firmware.a \
    build/obj/heartfull_membrane.o \
    build/obj/bzy_mpda.o \
    build/obj/tripartite_discriminant.o
```

Or use the Makefile (Linux/WSL only):

```bash
make all            # build everything
make boot           # assemble boot sector only
make firmware       # build C shared library + archive
make firmware-cpp   # build C++ wrapper
make run            # boot with QEMU
make verify         # run NSIGII verification checks
make clean          # remove build artifacts
```

5. Run the C# compositor

```powershell
# Requires .NET 8 SDK: https://dot.net

# Development mode (no QEMU needed — simulates boot PASS)
dotnet run --project mmuko-compositor.csproj -- --simulate-pass

# Explicit states
dotnet run --project mmuko-compositor.csproj -- --boot-passed true --tier1 yes --tie

# HOLD scenario (T1 pending)
dotnet run --project mmuko-compositor.csproj -- --boot-passed true --tier1 maybe --t

# ALERT scenario (T1 violated)
dotnet run --project mmuko-compositor.csproj -- --boot-passed true --tier1 no --tier
```

File reference

File	Language	Role
`boot.asm`	NASM x86-16	FAT12 ring boot sector. Six-phase NSIGII calibration in assembly. Exactly 512 bytes.
`heartfull_firmware.h`	C	All types: `TrinaryState`, `MembraneOutcome`, `PerspectiveMembrane`, `QubitCompass`, `MaslowNeedsState`, `EnzymeOp`, `KanbanTrack`.
`heartfull_membrane.c`	C	Six-phase NSIGII calibrator. Trinary composition, enzyme degradation, compass rotation, drift theorem, membrane gate.
`bzy_mpda.h` / `bzy_mpda.c`	C	Byzantine Maybe PDA — formal 5-tuple $M=(Q,\Sigma,\Gamma,\delta,q_0,F)$, magnetic transition table, pushdown stack, LTCodec reverse-read.
`tripartite_discriminant.h` / `.c`	C	$G=\{U,V,W\}$ discriminant $\Delta=b^2-4ac$, Byzantine fault detection, quadratic roots (BUILD/BREAK paths).
`nsigii_cpp_wrapper.cpp`	C++17	RAII wrappers: `Trinary` with `operator*`, `MembraneCalibrator`, `ByzantineChecker`,

File	Language	Role
		`MPDARunner`, `DriftMonitor`. C-linkage exports for P/Invoke.
`NSIGII_HeartfullFirmware.cs`	C#	Compositor. Boot-gate enforced via `HeartfullFirmware.Create(bootPassed)`. Implements all six phases, RIFT trinary, enzymes, Kanban three-track, P/Invoke bridge.
`NSIGII_HeartFeltFirmware.cs`	C#	UI layer. Refuses to construct if membrane is HOLD or ALERT. Renders Kanban board, discriminant panel, trinary state display.
`Main.cs`	C#	LTE entry point. Parses args, runs NSIGII, demos trinary/enzyme/discriminant/drift.
`mmuko-compositor.csproj`	MSBuild	.NET 8 project file. References native library via P/Invoke.
`Makefile`	GNU Make	Full build orchestration for Linux/WSL.
`boot.bin`	Binary	Pre-assembled boot sector (512 bytes).
`mmuko-os.img`	Binary	Pre-imaged 1.44 MB FAT12 disk image.

Architecture

```
Hardware / BIOS
      |
boot.asm  (NASM, 16-bit)
  Phase N — Need-state init    (theta=0,   THERE_AND_THEN)
  Phase S — Safety scan        (theta=0,   T2 check)
  Phase I — Identity calib     (theta=120, HERE_AND_NOW)
  Phase G — Governance check   (theta=240, WHEN_AND_WHERE)
  Phase I — Internal probe P_I (compose alpha x beta x gamma)
  Phase I — Integrity / delta  (discriminant >= 0 ?)
      |
  MEMBRANE_PASS = 0xAA
      |
C firmware library (libnsigii_firmware.so / .a)
  heartfull_membrane.c     -- calibration engine
  bzy_mpda.c               -- Byzantine Maybe pushdown automaton
  tripartite_discriminant.c -- G={U,V,W} fault detection
```

```
      |
C++ wrapper (libnsigii_firmware_cpp.so)
  nsigii_cpp_wrapper.cpp  -- RAII + DriftMonitor + C exports
      |
C# compositor (.NET 8)
  NSIGII_HeartfullFirmware.cs  -- firmware logic + P/Invoke
  NSIGII_HeartFeltFirmware.cs  -- UI (loads only after PASS)
  Main.cs                      -- LTE entry point
```

LTF — Linkable Then Executable

This project uses the **LTF (Linkable Then Format)** pipeline. Files are linked before they are permitted to execute. The C# compositor will not run until `HeartfullFirmware.Create(bootPassed: true)` is called — the boot gate is the precondition. In production, the assembly writes `OUTCOME_PASS = 0xAA` to a shared memory location that the compositor reads before constructing.

Trinary alphabet

The firmware operates on a **four-value trinary alphabet**, not the classical binary:

Symbol	Value	Meaning	Action
`YES`	`+1`	Needs met, contract honoured	Proceed
`NO`	`0`	Needs violated, contract breached	ALERT
`MAYBE`	`-1`	Needs uncertain, response delayed	Enzyme pathway
`MAYBE_NOT`	`-2`	Deferred — do NOT handle for operator	System absorbs

RIFT trinary composition rules

```
YES   * YES       = YES        (both confirmed)
NO    * anything  = NO         (NO absorbs)
MAYBE * MAYBE     = YES        (double negation resolves)
MAYBE * YES       = MAYBE      (uncertainty persists)
MAYBE_NOT * any   = MAYBE_NOT  (defer wins)
```

Enzyme degradation — MAYBE states

When `MAYBE` is encountered on a Kanban thread, an enzyme operation is applied:

```
ENZYME_CREATE  : MAYBE -> YES    (creates a resolved state)
ENZYME_DESTROY : MAYBE -> NO     (destroys ambiguous state)
ENZYME_BUILD   : MAYBE -> YES    (builds toward resolution)
ENZYME_BREAK   : YES   -> MAYBE  (breaks apart certainty)
ENZYME_RENEW   : *     -> MAYBE  (refreshes to pending)
ENZYME_REPAIR  : NO    -> MAYBE  (patches violation back to pending)
```

Byzantine discriminant — G = {U, V, W}

The tripartite discriminant detects adversarial interference in the constitutional relationship between operator (U), institution (V), and third-party/attacker (W):

```
Delta = b^2 - 4ac
  where b = U + V + W  (sum of trinary signals)
        a = 1, c = 1   (unit constitutional constants)

Delta > 0  -> STABLE   : two real roots, W is benign or absent
Delta = 0  -> CRITICAL : W is present and creating pressure
Delta < 0  -> FAULT    : W is actively disrupting U-V relationship
```

Examples:

U	V	W	b	Delta	Region
+1	+1	+1	+3	+5	STABLE
-1	-1	-1	-3	+5	STABLE (all MAYBE is not a fault)
0	0	0	0	-4	FAULT
+1	-1	0	0	-4	FAULT

Note: all-MAYBE (`b=-3`, `Delta=+5`) is **STABLE**, not a fault. MAYBE is the normal real-world state of an institution that has not yet responded. The system must handle it, not collapse it to failure.

Maslow-Kanban three-track interface

The firmware exposes its state to the user via three Kanban tracks:

Track	Maslow tiers	Condition
Track A — Foundation	T1 (physiological) + T2 (safety)	Always active — PASS or HOLD
Track B — Aspiration	T3–T5 (belonging, esteem, actualisation)	**Locked** until membrane issues PASS
Track W — Adversarial	W-actor G={U,V,W} monitor	Always active

Track B is unconditionally locked until T1 and T2 are both satisfied. You cannot self-actualise (build OBINexus, pursue PhD, deploy CORN) while the firmware is returning HOLD or ALERT on food, water, or shelter.

Three-ring qubit compass

The firmware's internal scanning mechanism uses three temporal frames:

```
Ring 1 (outer)  — THERE_AND_THEN  : theta = 0    (historical needs record)
Ring 2 (middle) — HERE_AND_NOW    : theta = 120  (current scan)
Ring 3 (inner)  — WHEN_AND_WHERE  : theta = 240  (predicted state)
```

Coherence is maximum when all three rings align (`delta_theta = 0`). Drift is the measure of how far apart they are.

Drift theorem

Radial and angular drift between two tripolar scan vectors:

```
V(t) = (alpha, beta, gamma)         -- tripolar observation vector
D(t) = dV(t)/dt                     -- drift vector

Dr = ||V_t|| - ||V_{t-1}||          -- radial drift
  Dr > 0 : needs diverging (resolving)
  Dr < 0 : needs converging (approaching satisfaction)

omega = d(theta)/dt                  -- angular drift
  Weighted observation: W(t) = (2/3)*P(t) + (1/3)*P(t-1)
```

NSIGII six-phase identifier

Position	Letter	NATO	Firmware role
1	N	November	Need-state initialisation
2	S	Sierra	Safety scan
3	I	India	Identity calibration (Uche/Obi/Eze tripolar)
4	G	Golf/Gold	Governance layer (OHA/IWU/IJI)
5	I	India	Internal probe (P_I activation)
6	I	India	Integrity verification (discriminant check)

Requirements

Component	Requirement
`boot.asm`	NASM 2.14+
`*.c` / `*.h`	GCC 9+ or Clang 10+ with C11
`nsigii_cpp_wrapper.cpp`	GCC 9+ or Clang 10+ with C++17
`*.cs` / `*.csproj`	.NET 8 SDK
QEMU	qemu-system-x86_64
Makefile	GNU Make (Linux/WSL)

Igbo ontological framework

This system is grounded in the Igbo tripartite model of personhood and governance:

- **Uche / Obi / Eze** — mind / heart / will (identity calibration, Phase I)
- **OHA / IWU / IJI** — community / law / execution (governance layer, Phase G)

These are not decorative labels. They are the structural basis for the tripolar pointer algebra (`alpha=WANT`, `beta=NEED`, `gamma=SHOULD`) and the three-ring qubit compass.